■ 2.2.14 Advanced Topic: Controlling the Action of Rules

When you make a replacement with *expr* `/.` *rules*, each of the rules is tried *only once*.

Sometimes you will need to go on trying rules over and over again, until the expression you are working on no longer changes. You can do this using *expr* `//.` *rules*. You can think of the notation `//.` as representing an extension of the standard `/.` notation for replacements.

The `/.` operator applies the rule only once.

```
In[1]:= f[5] /. f[x_] -> x f[x-1]
Out[1]= 5 f[4]
```

`//.` goes on trying to apply the rules in order, until none of them apply any more.

```
In[2]:= f[5] //. {f[1] -> 1, f[x_] -> x f[x-1]}
Out[2]= 120
```

Both *expr* `/.` *rules* and *expr* `//.` *rules* try to apply each rule to *every part* of the expression *expr*.

Sometimes, you may need to apply rules only to a specific part of an expression. `Replace[`*expr*`,` *rules*`]` tries to apply *rules* to the whole of *expr*; it does not apply the rules to any of the subparts of *expr*.

You can use `Replace`, together with `Map` and `MapAt`, to control exactly which parts of an expression a replacement is applied to.

The operator `/.` tries to apply rules to all the parts of an expression.

```
In[3]:= f[x]^2 /. f[x] -> a
                 2
Out[3]= a
```

`Replace` applies rules only to the whole expression.

```
In[4]:= Replace[f[x]^2, f[x]^2 -> b]
Out[4]= b
```

No replacement is done here.

```
In[5]:= Replace[f[x]^2, f[x] -> a]
                   2
Out[5]= f[x]
```

expr `/.` *rules*	apply *rules* to all parts of *expr* just once
expr `//.` *rules*	apply *rules* to all parts of *expr* repeatedly, until the result no longer changes
`Replace[`*expr*`,` *rules*`]`	apply *rules* just once, to the whole of *expr* only

Types of replacement.